

PROYECTO NUEVO AVATAR V2

Programación V

Colegio Universitario de Cartago

Prof. Ing. Gabriel González Solano

Administración de academia

Objetivo general proyecto

Realizar la experiencia de la elaboración de un proyecto de programación pasando por todas las etapas del proceso de software.

Objetivos específicos

- Analizar requisitos de desarrollo de distintos componentes de software.
- Diseñar la propuesta de solución para los distintos componentes de software.
- Realizar la programación de los componentes de software que formarán parte de la solución.
- Probar los distintos componentes de software que formarán parte de la solución.
- Implementar los componentes de software que formarán parte de la solución.
- Trabajar en equipo para alcanzar la consecución de las metas del proyecto.
- Utilizar software que facilite la integración de código fuente y la gestión del ciclo de vida del software.
- Enfrentar incertidumbre en las historias de usuario.

Contexto del proyecto

Como es de conocimiento, se tiene una necesidad general en el CUC para muchas áreas relacionadas a Software que apoye la gestión de los diferentes departamentos. Departamentos como Recursos Humanos, entre otros requieren que se elaboren soluciones que faciliten a los colaboradores y clientes (estudiantes y profesores) sus actividades diarias y que los servicios que se brindan al estudiantado por medio de ellos sean fácilmente disponibles y accesibles para poder utilizarlos. En ese sentido se propuso la construcción de una serie de herramientas y servicios que resuelvan esta necesidad y se propone que durante el curso de programación V sea atendida.

Arquitectura general del sistema

A continuación, se presenta un vistazo arquitectónico de alto nivel de cada uno de los componentes que formarán parte de la solución a implementar:

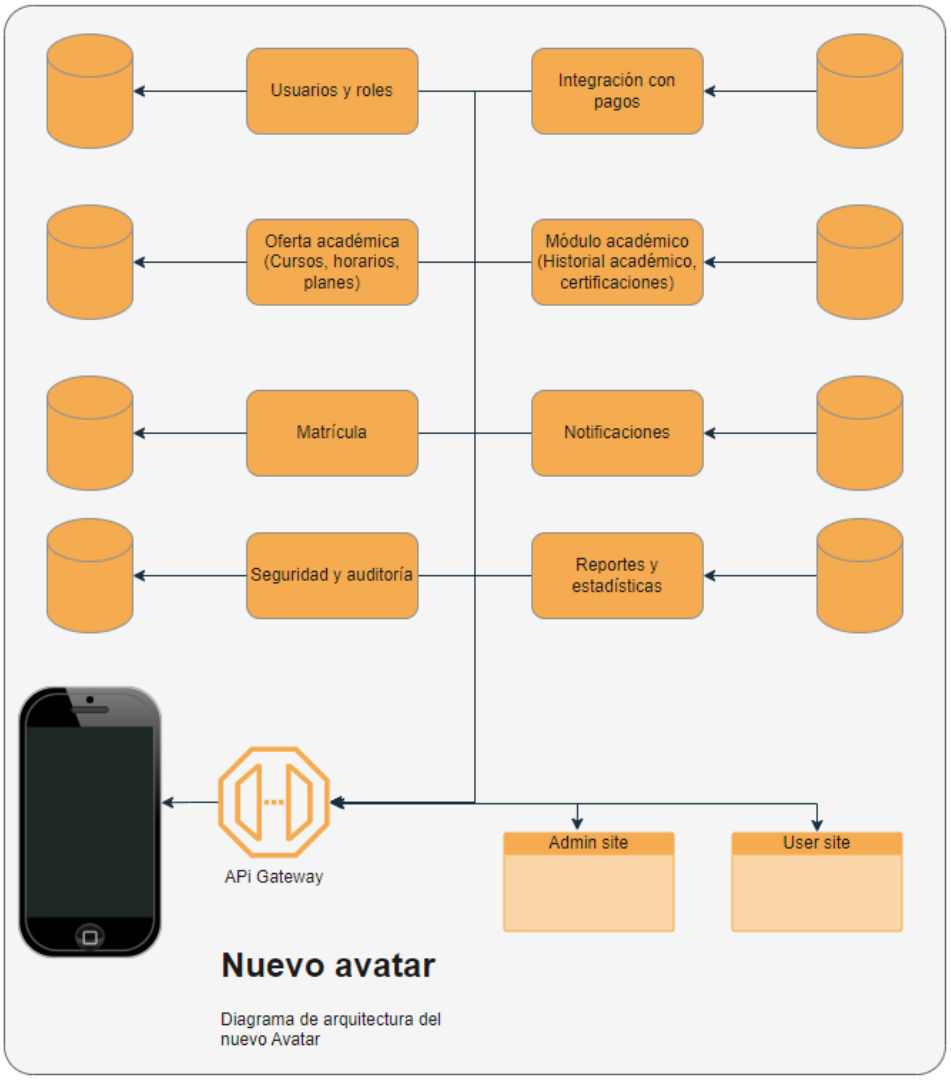


Ilustración 1 Arquitectura general

Organización del trabajo y de los equipos

Para trabajar el proyecto se seguirá el modelo de trabajo propuesto en Scrum. Dada la importante cantidad de requerimientos a atender, cada uno de los miembros realizará la atención de diferentes componentes.

Cada grupo se asignará un nombre por medio del cual será reconocido. Cada grupo de trabajo se distribuirá las historias de usuario que se le asignen entre sus integrantes. Si bien es cierto al final a responsabilidad es individual, cada

grupo es responsable de las historias que se le asignen para lograr un sprint exitoso.

Historias de usuario

Para la atención de los requisitos a desarrollar para el sistema, se crearán una serie de historias de usuario donde se detallará qué espera el usuario del sistema. A continuación, se realiza el detalle de las historias de usuario y sus criterios de aceptación.

Módulo de usuarios y roles

ID	Historia de usuario	Criterios de aceptación
USR1	Yo como arquitecto del sistema quiero un servicio para administrar usuarios	• Deben crearse 6 operaciones que permitan la administración de usuarios y que cumplan con el estándar Rest. Estas operaciones deben responder en el endpoint /usuario y deben permitir: ○ Crear un usuario. ○ Modificar un usuario. ○ Eliminar un usuario. ○ Obtener los datos de todos los usuarios. ○ Obtener los datos de un usuario por su llave primaria. ○ Obtener datos de los usuarios filtrados por: identificación, nombre, tipo. • Los datos que se requieren para un usuario son: ○ Email (será la identificación única del usuario). ○ Tipo de identificación. Debe permitirse que eventualmente existan más tipos de identificación.

		○ Identificación. ○ Nombre. ○ Rol del usuario. ○ Contraseña (debe almacenarse encriptada). • Debe validarse el formato del email. • Los emails deben ser de los dominios cuc.cr o cuc.ac.cr. • Si el email es del dominio cuc.cr, el rol de usuario debe ser estudiante. • Si el dominio es cuc.ac.cr, el rol de usuario debe ser profesor o administrador. • El nombre completo no puede ser vacío ni espacios en blanco. • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
USR2	Yo como arquitecto del sistema quiero un servicio para administrar roles	• Deben crearse 5 operaciones que permitan la administración de roles y que cumplan con el estándar Rest. Estas operaciones deben responder en el endpoint /rol y deben permitir sobre los roles: ○ Crear. ○ Modificar. ○ Eliminar.

		○ Obtener todos los datos. ○ Obtener los datos por llave primaria. • Los datos requeridos para un rol son: ○ Identificador del rol. ○ Nombre del rol. • Todos los datos son requeridos y no pueden ser vacíos ni espacios en blanco. • Debe validarse el nombre del rol que solo tenga letras y espacios. • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
USR3	Yo como arquitecto del sistema quiero un servicio para administrar parametrización	• Deben crearse 5 operaciones que permitan la administración de parámetros y que cumplan con el estándar Rest. Estas operaciones deben responder en el endpoint /parametro y deben permitir sobre los parametros: ○ Crear. ○ Modificar. ○ Eliminar. ○ Obtener todos los datos. ○ Obtener los datos por llave primaria. • Los datos requeridos para un parámetro son: ○ Identificador del parámetro.

		○ Valor del parámetro. • Todos los datos son requeridos y no pueden ser vacíos ni espacios en blanco. • En el caso del identificador del parámetro debe ser texto, de máximo 10 caracteres y solo permitir letras en mayúscula. • El valor debe ser texto, y formato libre, de máximo 500 caracteres. • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
USR4	Yo como arquitecto del sistema quiero un servicio para administrar módulos	• Deben crearse 5 operaciones que permitan la administración de módulos y que cumplan con el estándar Rest. Estas operaciones deben responder en el endpoint /modulo y deben permitir sobre los módulos: ○ Crear. ○ Modificar. ○ Eliminar. ○ Obtener todos los datos. ○ Obtener los datos por llave primaria. • Los datos requeridos para un módulo son: ○ Identificador del módulo. ○ Nombre del módulo.

		• Todos los datos son requeridos y no pueden ser vacíos ni espacios en blanco. • Debe validarse el nombre del módulo que solo tenga letras y espacios. • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
USR5	Yo como arquitecto del sistema quiero un servicio para login	• Se requiere de realizar un servicio rest que permita realizar las distintas operaciones de login del usuario, así como de la administración del token para realizar las operaciones. • En el endpoint /login se permitirá que el usuario pueda iniciar sesión, será una operación POST y recibirá en los headers el usuario y la contraseña. Debe verificar si coinciden el usuario y su contraseña. Tendrá el siguiente comportamiento: ○ Si la operación es exitosa el servicio debe responder un 201, junto con la siguiente estructura que permitirá realizar el resto de las operaciones. Este token debe tener una validez de 5 minutos por defecto, pero este periodo debe ser parametrizable.

{

expires_in = horaVencimiento,
access_token = jwtToken,
refresh_token = refreshToken,
usuarioID = identificador

}

- Si la operación falla el servicio debe responder un 401, junto con un mensaje que indique: “Usuario y/o contraseña incorrectos”.
- Todos los datos son requeridos y no pueden ser nulos o blancos.

- El refresh token, será un token temporal que permitirá extender el plazo de la sesión del usuario generando un nuevo token para las operaciones. Este token también tiene un periodo de expiración configurable, superior al token JWT.
- El endpoint /refresh permite obtener un nuevo token. Recibe como parámetro un refresh token y retorna los datos que se indican a continuación en formato json:

{

expires_in = horaVencimiento,
access_token = jwtToken,
refresh_token = refreshToken,

}

- Si el refresh token es válido se responderá con estado 201 y la estructura anterior. Si no es válido se responderá

		con un estado 401 con un mensaje que indique: “No autorizado”. • El endpoint /validate, recibe como parámetro un token y responderá con el código de estado 200 y un true si es válido. De lo contrario responderá con un 401 (Unauthorized).

Módulo de Oferta académica

ID	Historia de usuario	Criterios de aceptación
ACD1	Yo como arquitecto del sistema quiero un servicio para administrar instituciones	• Deben crearse 5 operaciones que permitan la administración de instituciones y que cumplan con el estándar Rest. Estas operaciones deben responder en el endpoint /institucion y deben permitir sobre las instituciones: ○ Crear. ○ Modificar. ○ Eliminar. ○ Obtener todos los datos. ○ Obtener los datos por llave primaria. • Los datos requeridos para una institución son: ○ Identificador de la institución. ○ Nombre de la institución. • Todos los datos son requeridos y no pueden ser vacíos ni espacios en blanco. • Debe validarse el nombre de la institución que solo tenga letras y espacios.

		• Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
ACD2	Yo como arquitecto del sistema quiero un servicio para administrar carreras	• Deben crearse 5 operaciones que permitan la administración de carreras y que cumplan con el estándar Rest. Estas operaciones deben responder en el endpoint /carrera y deben permitir sobre los roles: ○ Crear. ○ Modificar. ○ Eliminar. ○ Obtener todos los datos. ○ Obtener los datos por llave primaria. ○ Obtener las carreras por institución. • Los datos requeridos para una carrera son: ○ Identificador de la carrera. ○ Nombre de la carrera. ○ Institución en la que se brinda la carrera. ○ Director encargado. • Todos los datos son requeridos y no pueden ser vacíos ni espacios en blanco. • Debe validarse el nombre de la carrera que solo tenga letras y espacios.

		• El director de la carrera debe estar registrado como un profesor (ver servicio ACD6). • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
ACD3	Yo como arquitecto del sistema quiero un servicio para administrar cursos	• Deben crearse 6 operaciones que permitan la administración de cursos y que cumplan con el estándar Rest. Estas operaciones deben responder en el endpoint /curso y deben permitir sobre los cursos: ○ Crear. ○ Modificar. ○ Eliminar. ○ Obtener todos los datos. ○ Obtener los datos por llave primaria. ○ Obtener los cursos por carrera. • Los datos requeridos para un curso son: ○ Identificador del curso. ○ Carrera en la que se brinda. ○ Nivel en el que se brinda. ○ Nombre del curso. • Todos los datos son requeridos y no pueden ser vacíos ni espacios en blanco.

		• Debe validarse el nombre del curso que solo tenga letras y espacios. • El nivel es numérico y no debe ser menor a 1 ni mayor a 12. • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
ACD4	Yo como arquitecto del sistema quiero un servicio para administrar grupos	• Deben crearse 5 operaciones que permitan la administración de grupos y que cumplan con el estándar Rest. Estas operaciones deben responder en el endpoint /grupo y deben permitir sobre los grupos: ○ Crear. ○ Modificar. ○ Eliminar. ○ Obtener todos los datos. ○ Obtener los datos por llave primaria. • Los datos requeridos para un grupo son: ○ Identificador del grupo. ○ Número del grupo. ○ Curso al que se le asignará el grupo. ○ Profesor que lo impartirá. ○ Horario en qué se imparte ○ Cupo ○ Periodo

		• Todos los datos son requeridos y no pueden ser vacíos ni espacios en blanco. • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
ACD5	Yo como arquitecto del sistema quiero un servicio para administrar periodos	• Deben crearse 5 operaciones que permitan la administración de periodos y que cumplan con el estándar Rest. Estas operaciones deben responder en el endpoint /periodo y deben permitir sobre los periodos: ○ Crear. ○ Modificar. ○ Eliminar. ○ Obtener todos los datos. ○ Obtener los datos por llave primaria. • Los datos requeridos para un periodo son: ○ Año ○ Número de periodo ○ Fecha de inicio. ○ Fecha de fin. • Todos los datos son requeridos y no pueden ser vacíos ni espacios en blanco. • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio

		descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
ACD6	Yo como arquitecto del sistema quiero un servicio para administrar profesores	• Deben crearse 5 operaciones que permitan la administración de profesores y que cumplan con el estándar Rest. Estas operaciones deben responder en el endpoint /profesor y deben permitir sobre los profesores: ○ Crear. ○ Modificar. ○ Eliminar. ○ Obtener todos los datos. ○ Obtener los datos por llave primaria. • Los datos requeridos para un profesor son: ○ Número de identificación. ○ Tipo de identificación. ○ Email. ○ Nombre completo. ○ Fecha de nacimiento. ○ Teléfono(s) • Todos los datos son requeridos y no pueden ser vacíos ni espacios en blanco. • Debe validarse el nombre del profesor que solo tenga letras y espacios. • Debe ser mayor de edad. • Debe validar que el formato de email sea correcto y que pertenezca al dominio

		cuc.ac.cr (sin embargo, este dominio debe ser parametrizable). • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.

Módulo de Matrícula

ID	Historia de usuario	Criterios de aceptación
MAT1	Yo como arquitecto del sistema quiero un servicio para administrar pre matrícula	• Deben crearse 5 operaciones que permitan la administración de la prematricula de los estudiantes a cursos y que cumplan con el estándar Rest. Estas operaciones deben responder en el endpoint /prematricula y deben permitir: ○ Prematricular. ○ Modificar prematricula. ○ Eliminar prematricula. ○ Obtener todos los datos. ○ Obtener los datos por llave primaria. • Los datos requeridos para la prematricula son: ○ Identificación del estudiante. ○ Carrera a la que desea ingresar. ○ Cursos que desea prematricular.

		○ Observaciones ○ Periodo. • Deben ser cursos de primer nivel. • Las observaciones son opcionales, todos los demás datos son requeridos. • Solo se pueden prematricular periodos futuros (fecha de inicio posterior a la actual). • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
MAT2	Yo como arquitecto del sistema quiero un servicio para administrar matrícula	• Deben crearse 4 operaciones que permitan la administración de la matrícula de los estudiantes a cursos y que cumplan con el estándar Rest. Estas operaciones deben responder en el endpoint /matricula y deben permitir: ○ Matricular. ○ Modificar matricula. ○ Eliminar matricula. ○ Obtener los datos de estudiantes matriculados en un curso y grupo. • Los datos requeridos para la matricula son: ○ Identificación del estudiante. ○ Curso que desea matricular. ○ Grupo que desea matricular.

		○ Periodo. • Todos los datos son requeridos. • Solo se pueden matricular periodos activos (fecha de inicio posterior a la actual). • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
MAT3	Yo como arquitecto del sistema quiero un servicio para administrar expedientes de estudiantes	• Deben crearse 5 operaciones que permitan la administración de expediente y que cumplan con el estándar Rest. Estas operaciones deben responder en el endpoint /expediente y deben permitir sobre los estudiantes: ○ Crear. ○ Modificar. ○ Eliminar. ○ Obtener todos los datos. ○ Obtener los datos por llave primaria. • Los datos requeridos para un estudiante son: ○ Número de identificación. ○ Tipo de identificación. ○ Email. ○ Nombre completo. ○ Fecha de nacimiento.

		○ Dirección (provincia, cantón, distrito y otras señas) ○ Teléfono(s) • Todos los datos son requeridos y no pueden ser vacíos ni espacios en blanco. • Debe validarse el nombre del estudiante que solo tenga letras y espacios. • Debe validar que el formato de email sea correcto y que pertenezca al dominio cuc.cr (sin embargo, este dominio debe ser parametrizable). • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
MAT4	Yo como arquitecto del sistema quiero un servicio para administrar direcciones	• Deben crearse tres operaciones que permitan obtener los datos de provincias, cantones y distritos, que cumplan con el estándar rest. • Deben ser las siguientes: ○ /provincias: regresa un listado de todas las provincias registradas. ○ /cantones: recibe como parámetro una provincia y regresa todos los cantones de esa provincia.

		○ /distritos: recibe como parámetros una provincia y un cantón y regresa todos los distritos asociados. • Todos los parámetros indicados son requeridos. • En el caso de la operación de distritos, debe existir la correspondencia entre el cantón y la provincia, de lo contrario la consulta será incorrecta. • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
MAT5	Yo como arquitecto del sistema quiero un servicio para administrar notas	• Deben crearse las operaciones que permitan la administración de las notas de cursos obtenidas por estudiantes. • Deben ser las siguientes: ○ /cargardesglose: recibe el detalle del desglose de los rubros que se utilizarán para evaluar un curso. ○ /asignarnotarubro: asigna o modifica (depende del verbo) una nota a un rubro para un estudiante en un curso. ○ /obtenerdesglose: obtiene el desglose de rubros para un grupo de un curso en particular.

		○ /obtenernotas: obtiene el desglose de notas obtenidas por un estudiante para un curso en particular. • En el caso de cargar desglose, la sumatoria de los rubros debe sumar siempre 100. Si se carga nuevamente un desglose de rubros para un grupo, se elimina el anterior y se remplaza por el nuevo. • Una vez asignadas notas a estudiantes, no es posible modificar los rubros. • La asignación de las notas en un rubro debe tener un valor entre 1 y 100. • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.

Módulo de Académico

ID	Historia de usuario	Criterios de aceptación
ACA1	Yo como arquitecto del sistema quiero un servicio para consular el	• Se requiere de un servicio que regrese los promedios de notas obtenidos por un estudiante en todos los cursos que ha llevado. • Debe responder en el endpoint /historialacademico.

	historial académico	• Recibe como parámetro el tipo y la identificación del estudiante. • Debe regresar: ○ Código del curso. ○ Nombre del curso. ○ Promedio obtenido. • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
ACA2	Yo como arquitecto del sistema quiero un servicio para consultar un listado de estudiantes	• Se requiere de un servicio que regrese el listado de estudiantes que están matriculados en un periodo. • Debe responder en el endpoint /listadoestudiantes. • Recibe como parámetro el periodo que se desea consultar. • Debe regresar: ○ Llave (si no fuera la identificación). ○ Tipo de identificación. ○ Identificación. ○ Nombre completo. ○ Carrera. ○ Curso. ○ Grupo.

		• Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.

Módulo de integración de pagos y notificaciones

ID	Historia de usuario	Criterios de aceptación
IPN1	Yo como arquitecto del sistema quiero un servicio para administrar la facturación	• Se debe crear un servicio que permita la administración de la facturación realizada a partir de la matrícula. • Debe responder en el endpoint /factura. • Debe permitir: ○ Crear una factura. ○ Reversar (anular) una factura. ○ Obtener los datos de una factura en particular. ○ Obtener un listado de la facturación de un periodo en particular. • Para crear una factura se requiere: ○ Identificación del estudiante. ○ Monto por cobrar (para definir esto, vamos a establecer el costo del curso en 30.000 colones). • El detalle de la factura indicará: “Servicios estudiantiles”.

		• Se debe incluir un impuesto del 2% del monto por cobrar. • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Anotación técnica: debe usar el patrón de encabezado detalle. • Las facturas inicialmente se crean pendientes de cobro. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
IPN2	Yo como arquitecto del sistema quiero un servicio para administrar los pagos	• Se debe crear un servicio que permita la administración de los pagos realizados por las facturas de la matrícula. • Debe responder en el endpoint /pago. • Debe permitir: ○ Crear un pago que cancela una factura. ○ Reversar (anular) un pago. ○ Obtener un pago en particular. ○ Obtener un listado de pagos de un periodo en particular. • Al crear un pago la factura se marca como pagada. • Si se reversa se debe colocar de nuevo Pendiente de pago.

		• Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.
IPN3	Yo como arquitecto del sistema quiero un servicio para gestión de notificaciones	• Se debe crear un servicio que permita realizar notificaciones por medio de correo electrónico. • Debe responder en el endpoint /notificar. • Debe recibir como parámetros: ○ Email donde enviar la notificación. ○ Asunto de la notificación. ○ Cuerpo de la notificación (puede incluirse en formato HTML). • Los datos del servidor y cuenta de correo que se utiliza para envío de correos deben ser parametrizables. • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”. • Tomar en cuenta el registro de bitácoras, indicado en la sección Manejo de bitácoras y realizarlo por medio del servicio indicado en la HU GEN1.

Módulo de General

ID	Historia de usuario	Criterios de aceptación
GEN1	Yo como arquitecto del sistema quiero un servicio para registrar y consultar bitácoras	• Se debe crear un servicio que permita realizar el registro de bitácoras. • Debe responder en el endpoint /bitacora. • Debe recibir como parámetros: ○ Usuario que ejecuta la acción. ○ Descripción de la acción • Cuando realice el registro, además, debe incluir la fecha de la bitácora, la cual debe corresponder con la fecha/hora actual. • Todos los datos son requeridos y no puede ser vacío ni espacios en blanco. • Todas las operaciones requieren de un token de autorización de usuario autenticado para poder ejecutar, obtenido con el servicio descrito en la HU USR5. Se debe validar contra el método “validate”.

Módulo de administración

ID	Historia de usuario	Criterios de aceptación
ADM1	Yo como arquitecto del sistema quiero contar con una plantilla de sitio reutilizable para que todas las pantallas mantengan consistencia visual y de navegación.	• Debe existir un layout maestro con: encabezado (logo y título del sistema), menú lateral colapsable, área de contenido, barra de usuario (nombre, rol, botón de salir) y pie de página. • El menú se construye dinámicamente por rol (Administrador, Profesor, etc) usando la información de módulos/roles del backend.

		• Debe ser responsive (desktop, tablet, móvil). • Debe existir componente de notificaciones (éxito, error, warning, info). • Toda navegación debe preservar ruta activa, breadcrumbs y título de página. • Registrar en bitácora (GEN1): acceso al sistema, cambio de ruta (pantallas críticas) y errores de UI.
ADM2	Yo como usuario del sistema quiero iniciar sesión para acceder a las funcionalidades según mi rol	• Formulario con email (validación de formato y dominio permitido) y contraseña (requerida). • Al autenticar, consumir POST /login y guardar access_token, expires_in, refresh_token, usuarioID. • Si credenciales inválidas, mostrar mensaje “Usuario y/o contraseña incorrectos”. • Auto-redirigir al Dashboard de bienvenida tras login exitoso. • Soportar recordar sesión (opcional) y cerrar sesión que invalide tokens en cliente. • Al expirar el token, usar /refresh para renovar sesión de forma transparente; si falla, redirigir a login. • Registrar en bitácora: intento exitoso/erróneo de login, logout.
ADM3	Yo como usuario autenticado quiero una	• Mostrar saludo, rol, y atajos a funcionalidades frecuentes (según rol).

	pantalla de bienvenida con accesos rápidos y métricas para orientarme.	• Widgets mínimos: ○ Admin: #Usuarios, #Carreras, #Cursos, Periodos activos, Facturas pendientes. ○ Profesor: Mis grupos, Cursos que imparto en el periodo, Acceso a carga de desglose/notas. • Panel de notificaciones recientes (envíos del módulo IPN3) y últimas bitácoras relevantes. • Accesos directos a ayuda/soporte y perfil de usuario. • Registrar en bitácora la visualización del dashboard.
ADM4	Yo como administrador quiero administrar usuarios para mantener el control de acceso.	• Listado con columnas: Email (ID), Tipo de identificación, Identificación, Nombre, Rol. • Filtros: identificación, nombre, tipo, dominio (cuc.cr / cuc.ac.cr), rol. • Orden por columnas; paginación (configurable). • Crear/Editar (modal o página): • Validar email (formato + dominios permitidos: cuc.cr → rol Estudiante; cuc.ac.cr → Profesor/Administrador). • Nombre no vacío/solo espacios. • Contraseña: al crear es requerida; al editar es opcional; almacenamiento encriptado en backend.

		• Asignación de rol (lista de USR2/USR4). • Eliminar: confirmación; manejo de errores por dependencias (si aplica). • Ver detalle: incluye auditoría básica (última modificación). • Seguridad/Autorización: todas las llamadas con token válido (validado por /validate). • Bitácoras: alta, modificación, baja, consultas (detalle/listado/filtrado) con JSON del registro previo/actual.
ADM5	Yo como administrador quiero administrar roles para controlar permisos.	• CRUD de roles (ID y nombre con validación de solo letras y espacios). • Asignación de permisos de menú: matriz rol ↔ módulo/opción (toggle por item). • Persistencia en backend (módulos/menú), lectura al iniciar sesión para construir el menú (ADM1). • Validaciones: nombre requerido, único (manejo de error de duplicado). • Bitácoras: todas las operaciones; registrar los cambios de permisos (JSON diff).
ADM6	Yo como administrador quiero administrar parámetros del sistema para configurar reglas (p.	• CRUD con restricciones del backend: ○ ID: texto mayúsculas, máx. 10; Valor: texto libre, máx. 500. • Incluir paginación.

	ej., dominios válidos, expiraciones).	• Bitácoras: todas las operaciones; registrar los cambios de permisos (JSON diff).
ADM7	Yo como administrador quiero administrar los módulos visibles en el menú.	• CRUD de módulos: ID, nombre (solo letras y espacios), estado (activo/inactivo), orden. • Asociación módulo ↔ rol (similar a ADM5) para construir menú dinámico. • Bitácoras en todas las operaciones.
ADM8	Yo como administrador quiero una pantalla para administrar instituciones.	• Listado con búsqueda por nombre; CRUD con validación de solo letras y espacios. • Relaciones: impedir eliminación si hay carreras asociadas (mensaje claro). • Bitácoras en todas las operaciones
ADM9	Yo como administrador quiero una pantalla para administrar carreras por institución.	• Listado con filtro por institución; CRUD con validación de nombre (letras/espacios). • Director: debe existir como profesor (ACD6); selector con búsqueda. • Relaciones: impedir eliminación si existen cursos vinculados. • Bitácoras en todas las operaciones.
ADM10	Yo como administrador quiero administrar cursos.	• Listado con filtro por carrera y nivel; CRUD con validaciones: • Nivel entre 1 y 12; nombre con letras/espacios. • Relaciones: bloqueo de eliminación si hay grupos asociados. • Bitácoras en todas las operaciones.

ADM11	Yo como administrador quiero una pantalla para administrar profesores.	• ABM con validaciones: • Mayor de edad (fecha de nacimiento). • Email con dominio cuc.ac.cr (parametrizable). • Nombre solo letras/espacios; teléfonos requeridos (formato local). • Búsqueda por identificación/nombre/email; paginación y orden. • Relaciones: impedir eliminar si dicta grupos activos. • Bitácoras en todas las operaciones.
ADM12	Yo como administrador quiero una pantalla para administrar periodos.	• CRUD con: Año, Número de periodo, Fecha de inicio/fin (validación fin ⩾ inicio). • Estados derivados: futuro (inicio > hoy), activo (hoy entre inicio y fin), cerrado (fin < hoy). • Impedir solapamientos de periodos con mismo año/número. • Bitácoras en todas las operaciones.
ADM13	Yo como administrador/profesor quiero una pantalla para administrar grupos por curso/periodo.	• CRUD de grupos: número de grupo, curso, profesor, horario, periodo. • Validaciones: número único por curso/periodo; horario con formato estándar; profesor existente. • Relaciones: impedir eliminación si hay matrícula asociada. • Bitácoras en todas las operaciones.

ADM14	Yo como administrativo quiero una pantalla para visualizar y gestionar prematrículas de estudiantes.	• Listado filtrable por periodo (solo futuros), carrera, curso, estudiante. • CRUD conforme a reglas del backend; observaciones opcionales. • Validar que el periodo sea futuro; prevenir duplicados (estudiante-curso-periodo). • Bitácoras en todas las operaciones.
ADM15	Yo como administrativo quiero una pantalla matricular estudiantes en cursos/grupos de periodos activos.	• Flujo guiado: seleccionar estudiante → curso → grupo → confirmar. • Validar periodo activo y evitar duplicidad (misma combinación estudiante-curso-grupo). • Mostrar cupos (si backend lo provee) y estado por grupo. • Listado de matriculados por curso/grupo con filtros; exportable CSV. • Bitácoras en todas las operaciones.
ADM16	Yo como administrativo quiero una pantalla para administrar expedientes.	• CRUD: identificación, tipo, email (dominio cuc.cr parametrizable), nombre, fecha de nacimiento, teléfonos, dirección (provincia/cantón/distrito con validación de correspondencia), otras señas. • Servicios de dirección: /provincias, /cantones?provincia=, /distritos?provincia=&canton=;

		manejar interdependencias y limpieza de selects. • Bitácoras en todas las operaciones.
ADM17	Yo como profesor quiero una pantalla para cargar el desglose y asignar/modificar notas por rubro.	• Cargar desglose: suma de rubros = 100; si ya existía y no hay notas, reemplazar; si hay notas, bloquear edición de rubros. • Endpoint: /cargardesglose y /obtenerdesglose. • Asignar nota rubro: validar 1-100; crear/editar según verbo; vista por grupo y por estudiante. • Endpoints: /asignarnotarubro, /obtenernotas. • Estados: sin desglose (mensaje y CTA), sin notas, cierres de edición según reglas. • Bitácoras en todas las operaciones.
ADM18	Yo como administrativo/profesor quiero una pantalla para consultar promedios por estudiante.	• Búsqueda por tipo/identificación; resultado: código del curso, nombre del curso, promedio. • Filtros por periodo/año; exportar CSV/PDF. • Paginación y orden. • Bitácoras en todas las operaciones.
ADM19	Yo como administrativo quiero una pantalla para obtener listados por periodo.	• Filtro por periodo, carrera, curso, grupo; columnas: llave/ID, tipo, identificación, nombre, carrera, curso, grupo. • Exportar CSV/PDF; orden y paginación.

		• Bitácoras en todas las operaciones.
ADM20	Yo como administrativo quiero una pantalla para administrar facturas ligadas a matrícula.	• Crear factura: seleccionar estudiante/matrícula → calcular monto base 30 000 + impuesto 2% → encabezado-detalle visible (“ Servicios estudiantiles”). • Reversar factura (anular) con confirmación y motivo. • Listar por periodo/estudiante/estado; ver detalle. • Estados: pendiente al crear; cambiar según pagos. • Bitácoras en todas las operaciones.
ADM21	Yo como administrativo quiero una pantalla para registrar y consultar pagos.	• Registrar pago que cancela factura (validaciones de monto/estado). • Reversar pago con confirmación y motivo; actualizar estado de factura. • Listar pagos por periodo; ver detalle y comprobante (si aplica). • Bitácoras en todas las operaciones.
ADM22	Yo como administrativo quiero una pantalla para enviar notificaciones por correo.	• Formulario: email(s) validados, asunto, cuerpo (HTML permitido, sanitizado). • Confirmación previa; mostrar resultado de envío (éxito/error). • Historial de envíos (fecha, destinatarios, asunto, estado). • Parámetros de servidor/cuenta obtenidos de parámetros.
ADM23	Yo como administrador quiero	• Filtros: fecha (rango), usuario, acción, módulo; paginación y orden.

	una pantalla para consultar bitácoras para auditoría.	• Detalle: JSON de “antes/después” (según operación), con formateo legible. • Seguridad: acceso solo a administradores.
ADM24	Yo como usuario autenticado quiero una pantalla para ver mi perfil y cambiar contraseña.	• Perfil: ver datos básicos (solo lectura de campos no editables). • Cambiar contraseña: formulario con validaciones (longitud, complejidad configurable por parámetros). • Cerrar todas las sesiones (en cliente): limpiar tokens y requerir login nuevamente. • Bitácoras: cambios de contraseña y actualizaciones de perfil.

Módulo de Mobile

ID	Historia de usuario	Criterios de aceptación
MOB0	Yo como arquitecto del sistema quiero una plantilla móvil reutilizable para mantener consistencia visual y de navegación.	• La app debe tener layout con: barra superior (logo/título), bottom tab (Inicio, Mis cursos, Matrícula, Pagos, Perfil) y área de contenido. • Incluir componente de toasts/snackbars y diálogo de confirmación reutilizable.
MOB1	Yo como estudiante quiero iniciar sesión para	• Formulario con email (validación de dominio parametrizable, por defecto cuc.cr) y contraseña (no vacía).

	acceder a mis servicios.	• Consumir POST /login y almacenar access_token, expires_in, refresh_token, usuarioID de forma segura (Keychain/Keystore). • Opción “Recordarme” y cerrar sesión que limpie credenciales locales. • Si credenciales inválidas: mostrar “Usuario y/o contraseña incorrectos”. • Registrar en bitácora: login ok/fallido y logout.
MOB2	Yo como estudiante autenticado quiero ver un resumen de mi estado para orientarme rápidamente.	• Mostrar tarjetas resumidas: Prematrículas activas, Matrícula del periodo, Cursos actuales, Facturas/Pagos pendientes. • Accesos rápidos a Prematrícula, Matrícula, Mis cursos, Pagos.
MOB3	Yo como estudiante quiero una pantalla para consultar mi expediente y actualizar datos permitidos.	• Mostrar datos personales; editables: teléfonos y dirección; no editables: identificación, email institucional. • Selectores jerárquicos /provincias, /cantones, /distritos con validación de correspondencia. • Confirmación de cambios. • Registrar en bitácora consultas y actualizaciones.
MOB4	Yo como estudiante quiero una pantalla para realizar mi prematrícula a	• Selector de carrera y cursos disponibles por periodo futuro. • Formulario: curso, periodo, observaciones. • Validar reglas del backend (no duplicados, periodo futuro).

	cursos de periodos futuros.	• Confirmar y mostrar resultado; opción de notificación por correo (IPN3). • Bitácora de operación y de consulta.
MOB5	Yo como estudiante quiero una pantalla para matricularme en un grupo de un curso en periodos activos.	• Flujo: elegir curso disponible → seleccionar grupo → confirmar. • Impedir duplicidad del mismo curso/grupo; validar periodo activo. • Confirmación, navegación a detalle de matrícula y registro en bitácora.
MOB6	Yo como estudiante quiero una pantalla ver mis cursos del periodo actual.	• Listado con búsqueda local y pull-to-refresh. • Acceso al detalle: profesor, horario, grupo, enlaces rápidos a Notas y Historial. • Bitácora de consulta.

ID	Historia de usuario	Criterios de aceptación
GTW1	Yo como arquitecto del sistema quiero configurar un Gateway como punto único de entrada para centralizar el acceso a los microservicios.	• El Gateway debe exponer un puerto único para recibir todas las peticiones de la Web y Apps. • Debe configurar el Routing (enrutamiento) para redirigir peticiones externas a los servicios internos correspondientes. • Debe soportar la redirección de los verbos HTTP (GET, POST, PUT, DELETE) según el estándar REST definido en las HU de servicios.

		• Ruta Admin: gateway/admin/{...} -> Microservicio Administrativo (/user, /rol, /parametro, /institucion). • Ruta Auth: gateway/auth/{...} -> Microservicio de Autenticación (/login, /refresh, /validate, etc). • Otras rutas, por ejemplo, matrícula: Gateway/mat/{…}, etc.
GTW2	Yo como arquitecto del sistema quiero implementar Autenticación y Autorización centralizada para proteger los microservicios.	• El Gateway debe interceptar todas las peticiones que requieran token de autorización (descritas en SRV1 a SRV19, excepto SRV7 y la operación de login y refresh). • Debe validar (operación validate) el JWT (access_token) antes de permitir el paso al microservicio destino. • Si el token es inválido o ha expirado, el Gateway debe responder inmediatamente con un código 401 Unauthorized.

Manejo de bitácoras

Cada acción importante (operación de CRUD) que realice un servicio debe ser registrada mediante una bitácora. Además, se deben registrar los errores técnicos que sucedan en el uso de los servicios. Los datos de las bitácoras son los siguientes:

- Fecha de la bitácora.
- Usuario que ejecuta la acción.
- Descripción de la acción (Acción realizada + JSON).

La descripción de la acción debe tener el siguiente detalle:

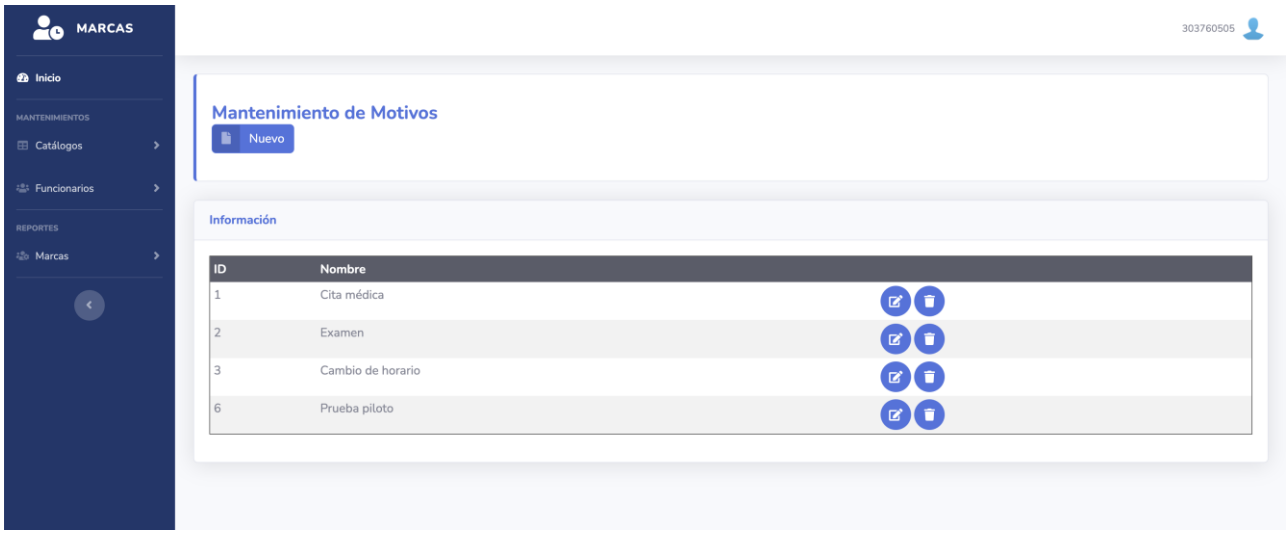
- En el caso de registros nuevos debe indicar el formato json el detalle del nuevo registro realizado.
- En el caso de actualización de registros, se debe indicar el formato json de la información del registro anterior y del registro actual.
- En el caso de eliminación de registros se debe indicar el formato json de la información eliminada.
- En el caso de consultas, únicamente se debe indicar “El usuario consulta <elemento>”.

Importante: las bitácoras se registran a través de llamadas al servicio respectivo. Debe llamarse desde los servicios que realizan la acción.

Flujo de pantallas

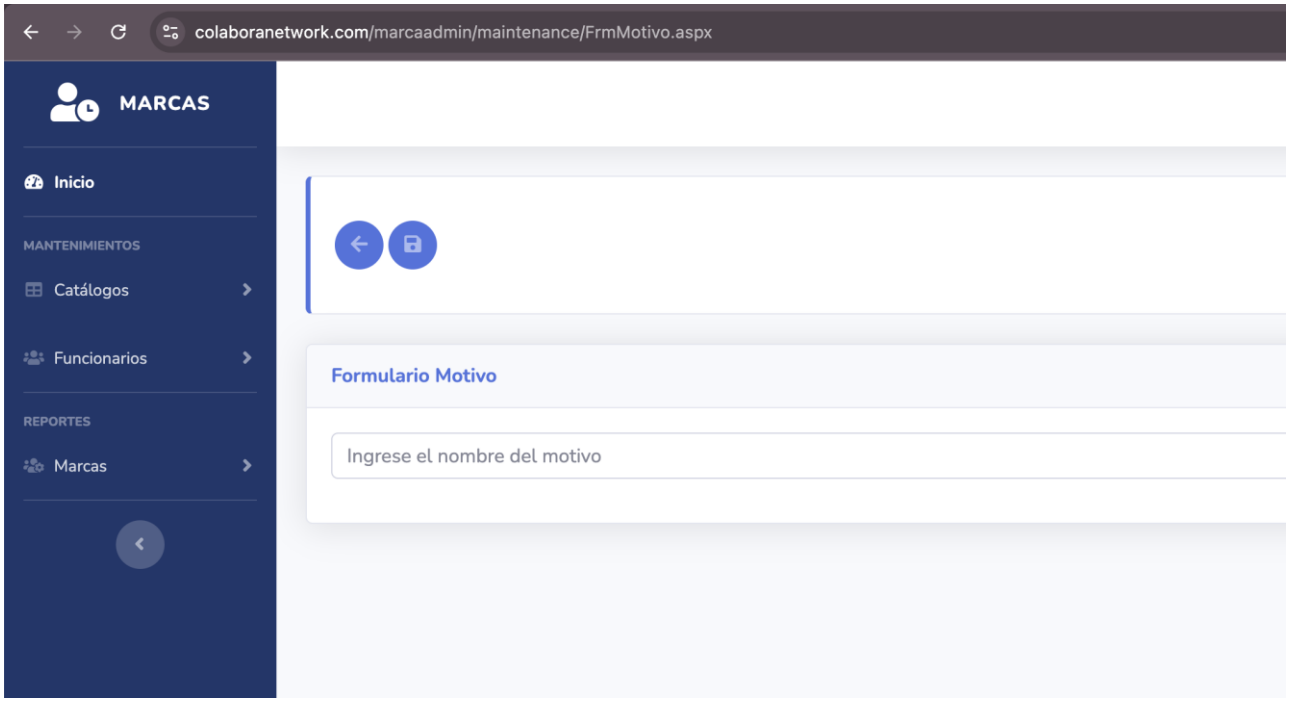
Con respecto al flujo que deben seguir las pantallas de mantenimientos deben seguir una dinámica similar a la siguiente:

- Se debe mostrar un listado de elementos correspondientes al mantenimiento, esta será la pantalla principal del mantenimiento, a la cual se ingresa justo al seleccionar la opción de menú:



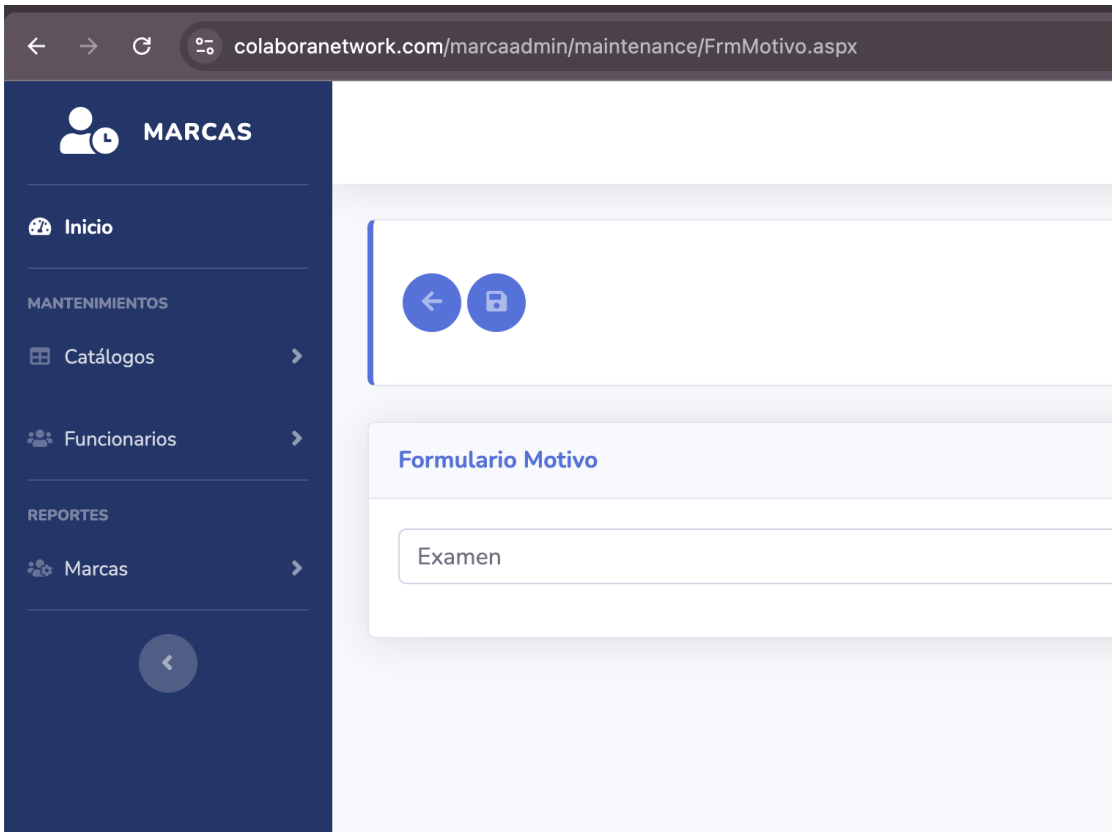
- El listado se debe mostrar siempre paginado, desplegando un máximo de 15 elementos por página. Este valor debe ser parametrizable.

- Para crear un nuevo elemento, debe existir un botón “Nuevo”, que, al presionarlo, lleva al formulario que permite ingresar los datos del nuevo elemento:



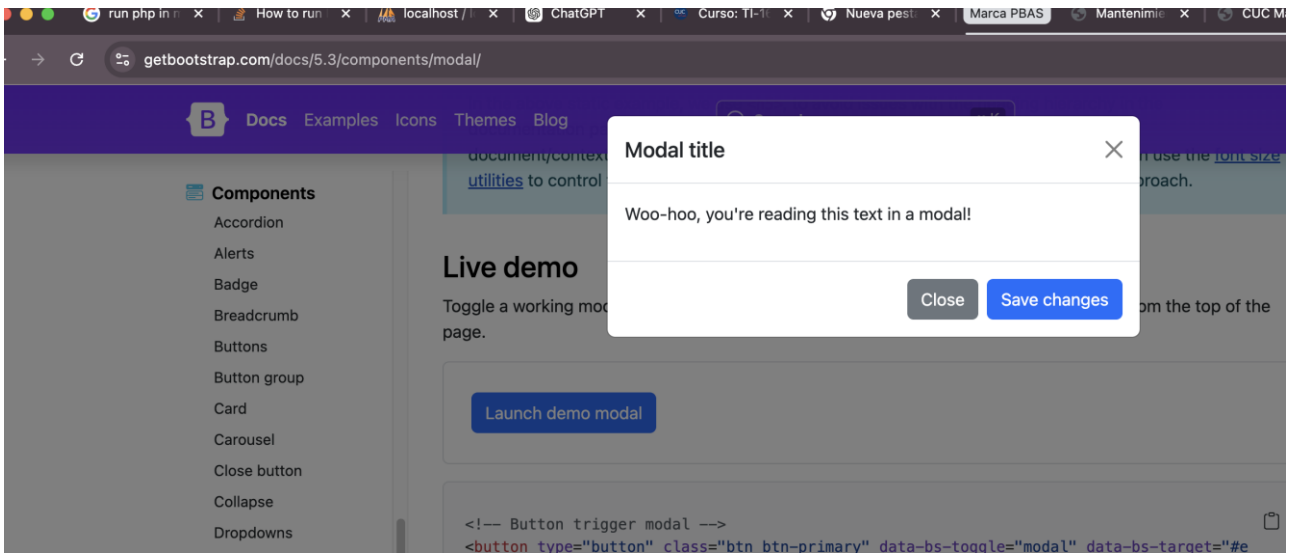
Este formulario además tendrá un botón para almacenar el nuevo elemento y uno para regresar al listado. Al usar el botón de guardar, si la operación es exitosa, debe regresar al listado y mostrar un mensaje de éxito, además, debe aparecer el nuevo elemento en el listado.

- Para editar cada fila del listado, debe tener un botón que ejecute esta acción, al presionarlo, debe redirigir a una pantalla similar a la de crear un nuevo elemento, pero con la información del elemento que se desea editar precargada, la cual podrá ser modificada y actualizada si se presiona el botón de salvar. De igual manera debe brindar la opción de regresar al listado principal.



- Finalmente, para eliminar, debe tener un botón que ejecute esta acción, al presionarlo, debe mostrar un diálogo de confirmación que indique: "¿Realmente desea eliminar el elemento seleccionado?" que indique dos opciones Si y No. Si selecciona "Si", se debe eliminar el elemento.

Respecto a los mensajes al usuario y cuadros de diálogo se deben mostrar al usuario similar a como se muestra el diálogo a continuación:



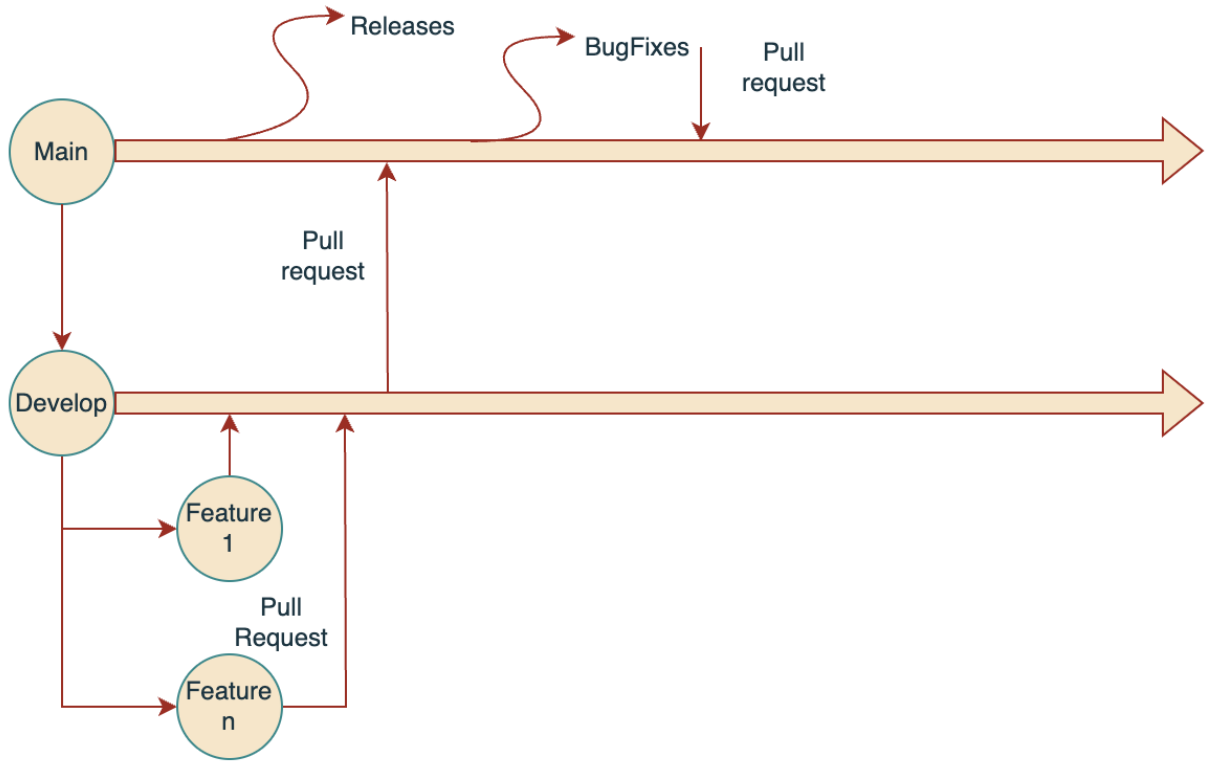
Administración de las Historias de Usuario

Las historias de usuario se cargarán en proyectos de alguna herramienta de administración de historias de usuario y seguimiento como Azure DevOps o GitHub.

Control del código fuente

El código fuente se administrará en una herramienta de repositorio de control de código fuente. En este caso se propone usar repositorio GIT ya sea en GitHub o en Azure Devops, durante el desarrollo del curso se valorará la mejor herramienta.

Se tendrá la siguiente estructura de ramas:



Creación y administración de cambios de la base de datos

Los equipos de trabajo definirán la base de datos, para esto en la segunda/tercera lección los distintos equipos formados deberán traer todo el problema analizado y presentar su propuesta de la base de datos. Las mejores

ideas de solución se tomarán para la base de datos que se usará en el proyecto.

Si posterior a la definición se necesita de un cambio justificado, debe gestionarse en el grupo de Teams del curso, de ser aprobado el que propone creará un script para el cambio y solicitará al profesor aplicar el cambio. Todos los equipos que se vea afectados por este cambio deben ajustar su código fuente para que maneje apropiadamente el cambio.

Tecnologías por utilizar

Para el primer alcance las tecnologías son las siguientes:

- Para la realización de los microservicios tiene la libertad de construirlos en la tecnología que considere conveniente su equipo de trabajo, siempre que se respeten los principios que se visualicen en clase. De igual forma puede elegir la tecnología de base de datos que más se adecue, siempre que mantenga la integridad de la información.

Para el segundo/tercer alcance las tecnologías son las siguientes:

- Los microservicios continuarlos en las tecnologías previamente elegidas tanto del servicio como tal, como la base de datos.
- Todo componente Web debe ser realizado en React o Blazor Web Assembly.
- Lo relacionado a móvil, debe realizarse en alguna tecnología multiplataforma o nativo (para Android).
- Gateway: debe realizarse en YARP u Ocelot.

Entregables del proyecto

1. Documentación de análisis y diseño (enfocada en las HU que debe atender el equipo).
    a. Portada.
    b. Introducción (Resumen del problema a resolver).

c. Diagrama de base de datos (completo). En caso de bases de datos Nosql, brinden una representación de las estructuras a almacenar.
d. Diagramas de casos de uso.
e. Diagramas de clases.
f. Pruebas técnicas.
g. Conclusiones y recomendaciones.
h. Bibliografía.

2. Script de base de datos.
3. Código fuente de los servicios rest, las Web y aplicación móvil.
4. Historias de usuario actualizadas.
5. Pull Request hacia la rama Main (o rama en GIT donde se encuentra la versión final de su código).

Con respecto a la documentación de pruebas técnicas, en esta actividad debe ejecutar pruebas sobre el servicio de tal forma que se cumpla con las funcionalidades y criterios de aceptación establecidos. Cada criterio de aceptación debe generar una prueba y debe documentarse de la siguiente manera:

# HU	Criterio de aceptación	Evidencia
ID de la HU respectiva	Descripción del criterio que se probó	Pantallazo con la ejecución exitosa.

Aspectos administrativos

1. El proyecto se elaborará en los equipos de trabajo definidos. La calificación será individual.
2. Debe dividir el proyecto en dos entregables, primero se realizará en la semana 10 y el segundo en la semana 14.
3. Valor del proyecto 15% (segundo entregable) y 30% tercer entregable.